

Homework 3. Classes and Lists

Due: Thursday, Feb 12, 11:55pm

How to turn in:

- Write several programs and submit them on the course website
- The names are in the assignment; incorrect names will be graded as 0

[**Note:** You will need `main()` methods to test your classes, which can be part of a `Main` class. But you do not need to submit `Main`. We will test your classes using our own `main()` method. You only need to submit the four classes listed explicitly in each part.]

Part A. Course Roster (15 points)

Write a program to define and use two classes: `Course` and `Student`.

A `Student` should have the following private attributes:

- `String name`
- `int id`

And the following public methods:

- A constructor that takes a name and an id
- Get methods, `getName()` to return name, and `getId()` to return id,
- Set method, `setName(String n)`, to update the name
- A `toString()` method to return a string with the name and id of the student
- An `equals(Student s)` method that checks whether the id of the current object is the same as the passed object. For example, calling `s1.equals(s2)` will return `true` if the id of `s1` and `s2` are the same and `false` otherwise.

The `Course` class should have the following private attributes:

- `String title`
- `int number`
- `String instructorName`
- An array of `Student` objects (up to 30) called `roster`
- The number of enrolled students

And the following public methods:

- A constructor that takes a course title, a course number, and a course instructor name
- Get methods, `getTitle()`, `getNumber()`, `getInstructorName()`, `getEnrollmentCount()`
- A `toString()` method to return a string with the course info **but without** info for each student
- An `equals(Course c)` method that compares the title and number (remember how to compare strings)

- `boolean add(Student s);`
A method to add a Student. It should verify that the student has not already been added by checking if the id is a repeat; if the student is already in the course or the course is full, return false
- `boolean remove(Student s);`
A method to remove a Student. If the student is in the course, it should remove them, shift the remaining students to the left as necessary, lower the number of enrolled students, and return true. If the student is not enrolled, return false
- `void print();`
A method to print out all of the course information, including a list of students.

Here is a sample test program:

```
Course c1 = new Course("Intro to Everything", 123, "Dr. Islam");
Student s1 = new Student("Jane", 1234);
Student s2 = new Student("Phillipe", 1235);
Student s3 = new Student("Lu", 1236);
Student s4 = new Student("Enrique", 1237);
c1.add(s1);
c1.add(s2);
c1.add(s3);
c1.add(s4);
c1.print();
c1.remove(s2);
c1.print();
```

A sample run of this program should look like this:

```
Course Title: Intro to Everything
Course Number: 123
Instructor: Dr. Islam
Enrollment: 4
    Name: Jane - ID: 1234
    Name: Phillipe - ID: 1235
    Name: Lu - ID: 1236
    Name: Enrique - ID: 1237
Course Title: Intro to Everything
Course Number: 123
Instructor: Dr. Islam
Enrollment: 3
    Name: Jane - ID: 1234
    Name: Lu - ID: 1236
    Name: Enrique - ID: 1237
```

Final Notes:

- Read the sample run carefully to understand the requirements of the problem correctly
- This program should be uploaded to the course website as separate `Student.java` and `Course.java` files

Part B. Basic List Operations (10 points)

Write a program to define and use a `NumberList` class. Do NOT start with existing code. The class should:

1. Contain an array with 10 positive integer values (10, 20, 30, 40, 50, 15, 25, 35, 45, 55)
2. Have an integer size that tells how many elements of the array are currently used
3. Have methods to
 - a. `void print()`, to print the list
 - b. `int findMaxIndex()` to find the index of the max value
 - c. `void deleteMax()`, to delete the max value by left-shifting
 - d. `boolean hasSpace()`, to check if there is space to append a number
 - e. `void appendNumber(int number)`, to append a number to the end of the list (if there is space)
 - f. `void reverse()`, to reverse the contents of the list

A sample run of your program should look like this:

```
List values (10): 10 20 30 40 50 15 25 35 45 55
This is a list of operations
0. Exit
1. Delete the max
2. Append a number
3. Reverse array
Enter your option: 3
List values (10): 55 45 35 25 15 50 40 30 20 10

Enter your option: 2
List is full, cannot append
List values (10): 55 45 35 25 15 50 40 30 20 10

Enter your option: 1
List values (9): 45 35 25 15 50 40 30 20 10

Enter your option: 1
List values (8): 45 35 25 15 40 30 20 10

Enter your option: 1
List values (7): 35 25 15 40 30 20 10

Enter your option: 1
List values (6): 35 25 15 30 20 10

Enter your option: 3
List values (6): 10 20 30 15 25 35

Enter your option: 1
List values (5): 10 20 30 15 25
```

```
Enter your option: 2
Enter the number to append: 30
List values (6): 10 20 30 15 25 30

Enter your option: 1
List values (5): 10 20 15 25 30

Enter your option: 2
Enter the number to append: 27
List values (6): 10 20 15 25 30 27

Enter your option: 2
Enter the number to append: 12
List values (7): 10 20 15 25 30 27 12

Enter your option: 0
Done!
```

Final Notes:

- Read the sample run carefully to understand the requirements of the problem correctly.
- If there are duplicated max numbers in the array, remove the first one
- This program should be uploaded to the course website as `NumberList.java`

Part C. Even More New List Operations (15 points)

Starting with your lab work (or the `List` class from the posted solution), add the following operations:

- `int getSize()`
This method returns the size of the list (`size`).
- `int getValue(int pos)`
This method returns the value of the element at the specified position. If the position is unused, it returns -1.
- `boolean hasDuplicates()`
This method checks if there are any duplications in the list, returning true if there are. For example, if a list contains 10, 30, and 20, the method should return false. However, if the list has 10, 30, 20, and 30, it will return true.
- `boolean isSorted()`
This method checks if the list is sorted in *ascending or descending* order. For example, if a list contains (10, 30, 20), the method should return false. However, if the list has (10, 20, 20, 30) or (30, 20, 10, 10, 5), it will return true. If the list is empty or has only one number, the method should return true

as well.

Final Notes:

- Read the instructions carefully to understand the requirements of the problem correctly
- This program should be uploaded to the course website as `List.java`